

UST ZC

CIE

# CIE 442 – Digital Signal Processing

## Project 1: Image Compression

### Prepared By

Abdelhady Mohamed 202201172  
Osama Mohamed 202201196

### Instructor

Dr.Mahmoud Abdelaziz

22 November 2025

# Contents

|          |                                                                    |          |
|----------|--------------------------------------------------------------------|----------|
| <b>1</b> | <b>Concept Explanation</b>                                         | <b>2</b> |
| 1.1      | Discrete Fourier Transform (DFT) . . . . .                         | 2        |
| 1.2      | Discrete Cosine Transform (DCT) . . . . .                          | 2        |
| 1.3      | Experimental Setup . . . . .                                       | 3        |
| 1.3.1    | Block Size . . . . .                                               | 3        |
| 1.3.2    | Retention Ratio . . . . .                                          | 3        |
| <b>2</b> | <b>Code Explanation</b>                                            | <b>4</b> |
| 2.1      | 2D-DFT & 2D-IDFT Implementation . . . . .                          | 4        |
| 2.1.1    | 2D-DFT . . . . .                                                   | 4        |
| 2.1.2    | 2D-IDFT . . . . .                                                  | 4        |
| 2.2      | 2D-DCT & 2D-IDCT Implementation . . . . .                          | 5        |
| 2.2.1    | Helper function to DCT & IDCT . . . . .                            | 5        |
| 2.2.2    | 2D-DCT . . . . .                                                   | 5        |
| 2.2.3    | 2D-IDCT . . . . .                                                  | 5        |
| 2.3      | Coefficient reduction . . . . .                                    | 6        |
| 2.4      | Main code . . . . .                                                | 6        |
| <b>3</b> | <b>Results and Discussion</b>                                      | <b>8</b> |
| 3.1      | MSE Table . . . . .                                                | 8        |
| 3.2      | MSE vs Retention Ratio . . . . .                                   | 9        |
| 3.3      | Effect of Block Size on Reconstruction Quality . . . . .           | 10       |
| 3.3.1    | Different block size and fixed Retention Ratio (0.25) . . . . .    | 10       |
| 3.3.2    | How The Block Size affect on Reconstruction Quality . . . . .      | 11       |
| 3.4      | Effect of Retention Ratio on Reconstruction Quality . . . . .      | 12       |
| 3.4.1    | Different Retention Ratio and Fixed Block Size (8*8) . . . . .     | 12       |
| 3.4.2    | How The Retention Ratio affect on Reconstruction Quality . . . . . | 13       |
| 3.5      | Why DCT is Better . . . . .                                        | 13       |
| 3.6      | Visual Comparison of Reconstructed Images . . . . .                | 13       |

# 1 Concept Explanation

## 1.1 Discrete Fourier Transform (DFT)

The Discrete Fourier Transform (DFT) is a concept in digital signal processing that helps us to convert a sequence of discrete time-domain samples into a set of frequency components by expressing the signal as a combination of complex sinusoids. The DFT changes an  $N$  point input signal into two  $N/2$  point output signals. And it's equations as follow:

$$1D - DFT \quad X(m) = \sum_{n=0}^{N-1} x(n) e^{-j\frac{2mn\pi}{N}} \quad (1)$$

$$1D - IDFT \quad x(n) = \frac{1}{N} \sum_{m=0}^{N-1} X(m) e^{j\frac{2mn\pi}{N}} \quad (2)$$

$$2D - DFT \quad X(m_1, m_2) = \sum_{c=0}^{N_2-1} \sum_{r=0}^{N_1-1} x(r, c) e^{-j2\pi\left(\frac{m_1 r}{N_1} + \frac{m_2 c}{N_2}\right)} \quad (3)$$

$$2D - IDFT \quad x(r, c) = \frac{1}{N_1 N_2} \sum_{m_2=0}^{N_2-1} \sum_{m_1=0}^{N_1-1} X(m_1, m_2) e^{j2\pi\left(\frac{m_1 r}{N_1} + \frac{m_2 c}{N_2}\right)} \quad (4)$$

## 1.2 Discrete Cosine Transform (DCT)

The Discrete Cosine Transform (DCT) is a special case from DFT, it uses only cosine functions and with the help of the property of even and symmetry of the cosine function it only produces real values which makes most of the signal into a low frequency components. And it's equations as follow:

$$1D - DCT \quad X(m) = \alpha(m) \sum_{n=0}^{N-1} x(n) \cos\left(\frac{\pi m(2n+1)}{2N}\right) \quad (5)$$

$$1D - IDCT \quad x(n) = \sum_{m=0}^{N-1} \alpha(m) X(m) \cos\left(\frac{\pi m(2n+1)}{2N}\right) \quad (6)$$

$$2D - DCT \quad X(m_1, m_2) = \alpha(m_1) \alpha(m_2) \sum_{c=0}^{N_2-1} \sum_{r=0}^{N_1-1} x(r, c) \cos\left(\frac{\pi m_1(2r+1)}{2N_1}\right) \cos\left(\frac{\pi m_2(2c+1)}{2N_2}\right) \quad (7)$$

$$2D - IDCT \quad x(r, c) = \sum_{m_1=0}^{N_1-1} \sum_{m_2=0}^{N_2-1} \alpha(m_1) \alpha(m_2) X(m_1, m_2) \cos\left[\frac{\pi m_1(2r+1)}{2N_1}\right] \cos\left[\frac{\pi m_2(2c+1)}{2N_2}\right] \quad (8)$$

Where  $\alpha(m)$  is a normalization factor:

$$\alpha(m) = \begin{cases} \sqrt{\frac{1}{N}}, & m = 0, \\ \sqrt{\frac{2}{N}}, & m > 0. \end{cases} \quad (9)$$

## 1.3 Experimental Setup

### 1.3.1 Block Size

In our project we want to transform the image into blocks to do our operation on it and this help us very much. It has a lot of advantages like small block size reduce the cost of matrix multiplications. We expect that the larger block size the higher quality and resolution but with higher computational load. The block sizes we use in our project are (4\*4, 8\*8, 16\*16, 32\*32, 64\*64, 128\*128, 256\*256).

### 1.3.2 Retention Ratio

After each block is transformed using DFT or DFT, we sort the magnitude of each block and take only largest N of magnitude based on used retention ratio and set the rest to be zero. This process helps us the blocks and make it faster to do the inverse operations on it IDFT or IDCT. The retention ratio we used are (0.1, 0.25, 0.5, 0.75, 1) and we expect that the higher retention ratio the higher quality images but with lower compression.

## 2 Code Explanation

### 2.1 2D-DFT & 2D-IDFT Implementation

#### 2.1.1 2D-DFT

---

```
1 %% 2D-DFT
2 function F = DFT_2D(Img)
3
4 [N1, N2] = size(Img);
5 n1 = 0:N1-1; m1 = n1.';
6 n2 = 0:N2-1; m2 = n2.';
7 W1 = exp(-1i*2*pi*(m1 * n1) / N1);
8 W2 = exp(-1i*2*pi*(m2 * n2) / N2);
9 F = W1 * Img * (W2)';
10 end
11 end
```

---

I have used a different method of implementation which is based on matrix multiplication instead of nested loop method. In this way I have two matrices which are  $W_1$  which represent all vertical components and  $W_2$  which represent all horizontal components then I use this formula to do the 2D-DFT:

$$F = W_1 \text{Img} W_2^T \quad (10)$$

#### 2.1.2 2D-IDFT

---

```
1 %% 2D-IDFT
2 function Img_rec = IDFT_2D(F)
3
4 [N1, N2] = size(F);
5 n1 = 0:N1-1; m1 = n1.';
6 n2 = 0:N2-1; m2 = n2.';
7 W1 = exp(-1i*2*pi*(m1 * n1) / N1);
8 W2 = exp(-1i*2*pi*(m2 * n2) / N2);
9 Img_rec = (W1' * F * (W2)) / (N1 * N2);
10 end
```

---

Using the same concept of matrix multiplication in IDFT with the inverse formula:

$$\text{Img} = \frac{W_1^T F W_2}{N_1 N_2} \quad (11)$$

## 2.2 2D-DCT & 2D-IDCT Implementation

### 2.2.1 Helper function to DCT & IDCT

---

```
1 %% Helper function to DCT & IDCT
2 function C = dct_matrix(N)
3 alpha = sqrt(1/N) * ones(N,1);
4 alpha(2:end) = sqrt(2/N);
5 n = 0:N-1;
6 C = zeros(N,N);
7 for k = 0:N-1
8     C(k+1, :) = alpha(k+1) * cos( pi*(2*n+1)*k / (2*N) );
9 end
10 end
```

---

The aim of this function is to create an  $N \times N$  orthonormal basis matrix, which is required to make DCT and IDCT in the matrix form, each row show a cosine basis matrix scaled by the normalization factor.

### 2.2.2 2D-DCT

---

```
1 %% 2D-DCT
2 function D = DCT_2D(Img)
3 Img = double(Img);
4 [N1, N2] = size(Img);
5 C1 = dct_matrix(N1);
6 C2 = dct_matrix(N2);
7 D = C1 * Img * C2.';
8 end
```

---

We do the 2D-DCT in same way like 2D-DFT but with the formula of the 2D-DCT:

$$D = C_1 \text{Img} C_2^T \quad (12)$$

### 2.2.3 2D-IDCT

---

```
1 %% 2D-IDCT
2 function Img_rec = IDCT_2D(D)
3
4 [N1, N2] = size(D);
5 C1 = dct_matrix(N1);
6 C2 = dct_matrix(N2);
7 Img_rec = C1.' * D * C2;
8 end
```

---

Same as before but inverse:

$$\text{Img} = C_1^T D C_2 \quad (13)$$

## 2.3 Coefficient reduction

---

```
1  %% Coefficient reduction
2  function reduction_out = Coefficient_Reduction(reduction_in, retention_ratio)
3
4  if retention_ratio >= 1
5      reduction_out = reduction_in;
6      return;
7  end
8  vec = abs(reduction_in(:));
9  num_to_keep = max(1, round(retention_ratio * numel(vec)));
10
11 if num_to_keep == numel(vec)
12     reduction_out = reduction_in;
13     return;
14 end
15 sorted_vec = sort(vec, 'descend');
16 kepted = sorted_vec(num_to_keep);
17 reduction_out = reduction_in .* (abs(reduction_in) >= kepted);
18 end
```

---

The aim of this function is to compares the matrix by retention ratio which is given. First, we transfer the block into long vector then sort it in descend and calculate the number of values that we will keep based on the retention ratio and finally set the unwanted values with zeros.

## 2.4 Main code

---

```
1  %% prepare the image
2
3  img = imread('cameraman.tif');
4
5  if ndims(img) == 3
6      img = rgb2gray(img);
7  end
8
9  img = imresize(img , [256 256]);
10 img = im2double(img);
11
12 %% Main code
13
14 block_sizes = [4, 8, 16, 32, 64, 128, 256];
15 retention_ratios = [0.10, 0.25, 0.50, 0.75, 1];
16
17 MSE_DCT_results = zeros(numel(block_sizes), numel(retention_ratios));
18 MSE_DFT_results = zeros(numel(block_sizes), numel(retention_ratios));
19
20 [R, C] = size(img);
```

---

First we start with prepare our image, resize it, make it gray scaled image and convert it to double values so that we can do any operation on it then we define our main parameters which are the block sizes, retention ratios, size of the image and two matrices to save all the values of MSE of DFT and DCT.

---

```

1 for bs = 1:numel(block_sizes)
2     block_size = block_sizes(bs);
3     fprintf('Running block size = %d\n', block_size);
4
5     for rr = 1:numel(retention_ratios)
6         retention_ratio = retention_ratios(rr);
7         fprintf('retention = %.2f', retention_ratio);
8

```

---

Nested for loops to try different block sizes and retention ratios

---

```

1     for r = 1:block_size:R
2         for c = 1:block_size:C
3             block = img(r:r+block_size-1, c:c+block_size-1);
4
5             % DCT
6             out_dct = DCT_2D(block);
7             out_dct = Coefficient_Reduction(out_dct, retention_ratio);
8             out_idct = IDCT_2D(out_dct);
9             Reconstructed_DCT(r:r+block_size-1, c:c+block_size-1) = out_idct;
10
11            % DFT
12            out_dft = DFT_2D(block);
13            out_dft = Coefficient_Reduction(out_dft, retention_ratio);
14            out_idft = IDFT_2D(out_dft);
15            out_idft = real(out_idft);
16            Reconstructed_DFT(r:r+block_size-1, c:c+block_size-1) = out_idft;
17        end
18    end

```

---

Now make another nested for loops to transfer the image into blocks and compute all the wanted operation on it (DFT, IDFT, DCT, IDCT, Coefficient\_Reduction)

---

```

1     mse_dct = mean((img - Reconstructed_DCT).^2, 'all');
2     mse_dft = mean((img - Reconstructed_DFT).^2, 'all');
3
4     MSE_DCT_results(bs, rr) = mse_dct;
5     MSE_DFT_results(bs, rr) = mse_dft;
6
7     compression_ratio = 1 / retention_ratio;

```

---

Finally, we calculate the MSE of each block with the given block size and retention ratio and save them in the matrices which we initiate them in the first. We use this formula to calculate the MSE:

$$\text{MSE} = \text{mean}((\text{image} - \text{Reconstructed Image})^2) \quad (14)$$



### 3 Results and Discussion

#### 3.1 MSE Table

| block_size | retention | DCT_MSE      | DFT_MSE      | compression_ratio |
|------------|-----------|--------------|--------------|-------------------|
| 4          | 0.10      | 4.823647e-04 | 6.849146e-04 | 10.000000         |
| 4          | 0.25      | 1.456788e-04 | 2.766151e-04 | 4.000000          |
| 4          | 0.50      | 2.439904e-05 | 8.183920e-05 | 2.000000          |
| 4          | 0.75      | 3.302464e-06 | 1.762705e-05 | 1.333333          |
| 4          | 1.00      | 1.893804e-32 | 5.846172e-33 | 1.000000          |
| 8          | 0.10      | 3.582255e-04 | 7.396529e-04 | 10.000000         |
| 8          | 0.25      | 8.360348e-05 | 2.425315e-04 | 4.000000          |
| 8          | 0.50      | 1.536279e-05 | 6.258857e-05 | 2.000000          |
| 8          | 0.75      | 1.698707e-06 | 1.172925e-05 | 1.333333          |
| 8          | 1.00      | 1.216619e-30 | 1.428622e-31 | 1.000000          |
| 16         | 0.10      | 2.441383e-04 | 6.210517e-04 | 10.000000         |
| 16         | 0.25      | 6.902581e-05 | 1.935770e-04 | 4.000000          |
| 16         | 0.50      | 1.295170e-05 | 4.308579e-05 | 2.000000          |
| 16         | 0.75      | 1.339986e-06 | 7.537002e-06 | 1.333333          |
| 16         | 1.00      | 1.264814e-30 | 1.085775e-30 | 1.000000          |
| 32         | 0.10      | 2.635489e-04 | 5.426726e-04 | 10.000000         |
| 32         | 0.25      | 7.224976e-05 | 1.733593e-04 | 4.000000          |
| 32         | 0.50      | 1.341939e-05 | 4.082583e-05 | 2.000000          |
| 32         | 0.75      | 1.389284e-06 | 6.797846e-06 | 1.333333          |
| 32         | 1.00      | 4.648431e-30 | 3.669746e-30 | 1.000000          |
| 64         | 0.10      | 2.152392e-04 | 4.344597e-04 | 10.000000         |
| 64         | 0.25      | 6.433500e-05 | 1.365388e-04 | 4.000000          |
| 64         | 0.50      | 1.222177e-05 | 3.078001e-05 | 2.000000          |
| 64         | 0.75      | 1.256721e-06 | 5.172879e-06 | 1.333333          |
| 64         | 1.00      | 1.239754e-29 | 1.364648e-29 | 1.000000          |
| 128        | 0.10      | 2.163371e-04 | 3.405381e-04 | 10.000000         |
| 128        | 0.25      | 6.547239e-05 | 1.104163e-04 | 4.000000          |
| 128        | 0.50      | 1.255736e-05 | 2.633244e-05 | 2.000000          |
| 128        | 0.75      | 1.285426e-06 | 4.536022e-06 | 1.333333          |
| 128        | 1.00      | 4.430065e-29 | 4.376193e-29 | 1.000000          |
| 256        | 0.10      | 2.509978e-04 | 3.361164e-04 | 10.000000         |
| 256        | 0.25      | 7.617295e-05 | 1.153174e-04 | 4.000000          |
| 256        | 0.50      | 1.422156e-05 | 2.795808e-05 | 2.000000          |
| 256        | 0.75      | 1.472869e-06 | 4.738640e-06 | 1.333333          |
| 256        | 1.00      | 1.107830e-28 | 2.221368e-28 | 1.000000          |

Table 1: Compression results for DCT and DFT at various block sizes and retention levels.

### 3.2 MSE vs Retention Ratio

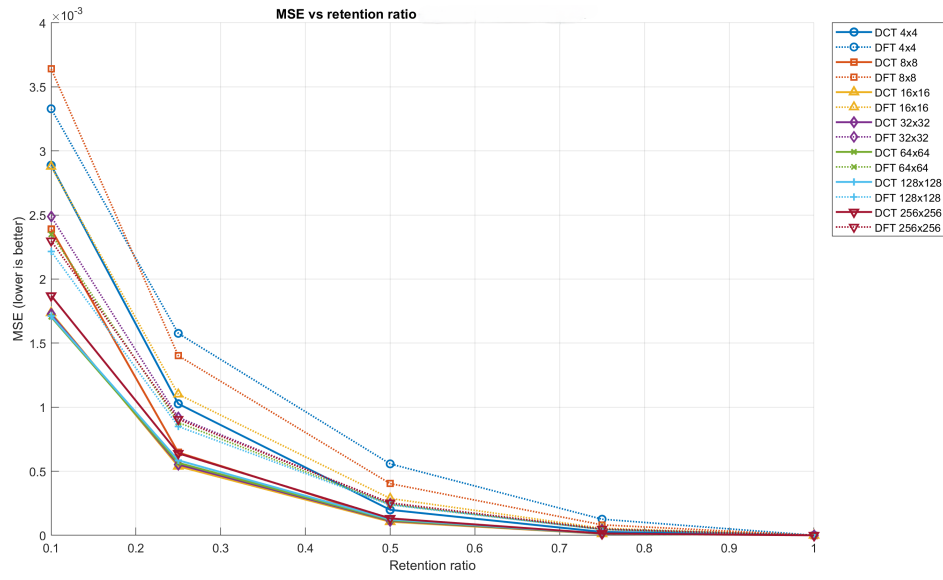


Figure 1: MSE vs Retention Ratio

### 3.3 Effect of Block Size on Reconstruction Quality

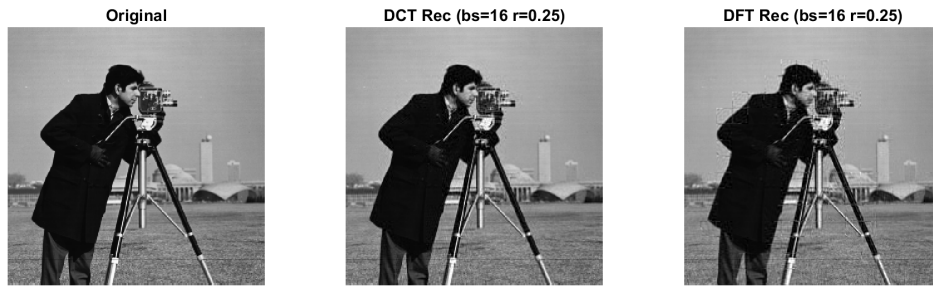
#### 3.3.1 Different block size and fixed Retention Ratio (0.25)



(a)Block size = 4



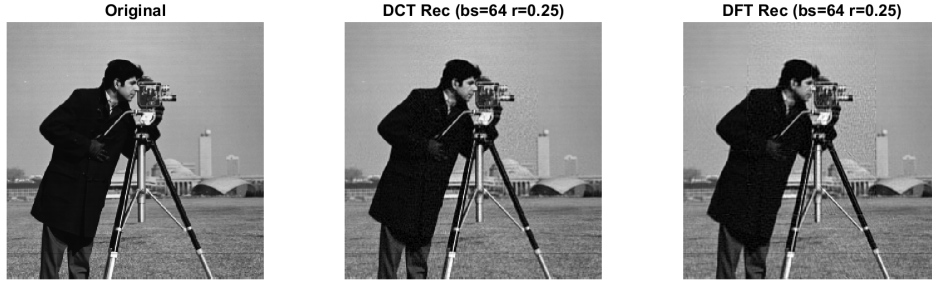
(b)Block size = 8



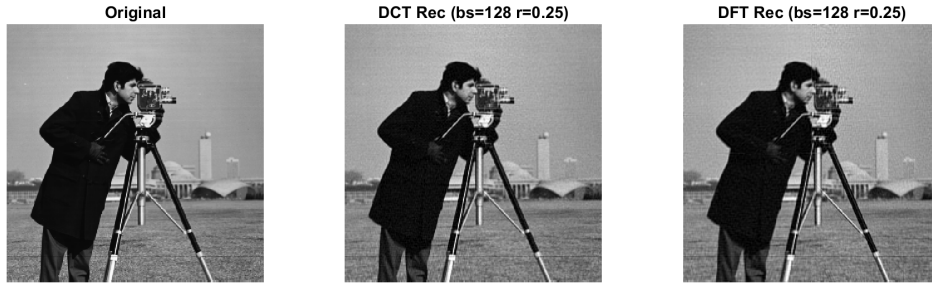
(c)Block size = 16



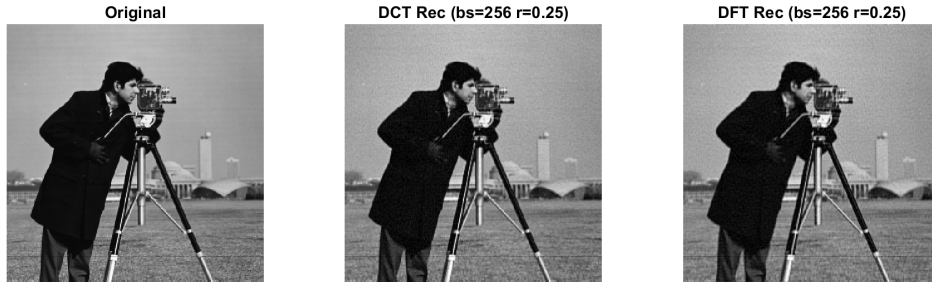
(d)Block size = 32



(e)Block size = 64



(f)Block size = 128



(g)Block size = 256

### 3.3.2 How The Block Size affect on Reconstruction Quality

As expected, when we increase the block size we get a low MSE based on the values from the MSE table and the given images with different block sizes.

### 3.4 Effect of Retention Ratio on Reconstruction Quality

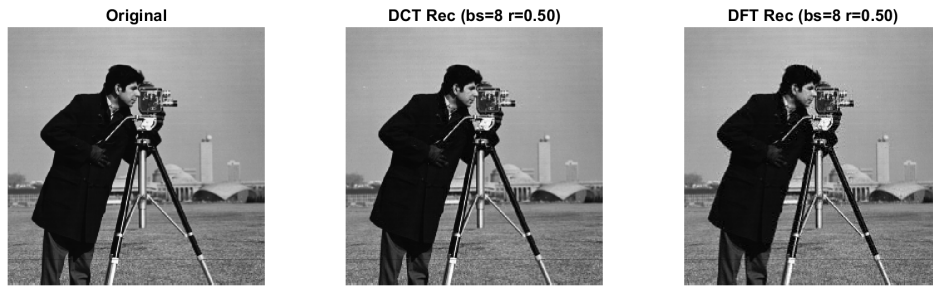
#### 3.4.1 Different Retention Ratio and Fixed Block Size (8\*8)



(a) Retention Ratio = 0.1



(b) Retention Ratio = 0.25



(c) Retention Ratio = 0.5



(d) Retention Ratio = 0.75



(e) Retention Ratio = 1.0

### 3.4.2 How The Retention Ratio affect on Reconstruction Quality

As expected, when we increase the retention ratio the MSE decreased because we keep more coefficients based on the values from the MSE table and the given images with different block sizes.

## 3.5 Why DCT is Better

For different block sizes and retention ratios, DCT always gives us a lower MSE than DFT this is because many reasons one of them is DFT have the energy across both real and imaginary parts in the other hand, DCT have only real parts so the energy compaction is better in DCT case

## 3.6 Visual Comparison of Reconstructed Images

If we take this image as an example to the real difference between DFT and DCT we will notice that if we look at the edges of the man coat we will notice that the transition from the lighter to darker color in the image which means there is a sudden change in frequency (high frequency area) in this case we can see the huge difference between DCT and DFT, the DCT take this sudden change in frequency and use it's property of symmetry it make this high frequency component into a low frequency by making this sudden change into a smother change, on the other hand the DFT just take it and does not make any changes of the sudden changes.



Block Size = 8, Retention Ratio = 1.0